EXPERIMENT 1

Student Name: Aditya Raj Bhakta **UID: 24BAI70782**
Branch: AIT-CSE (AIML) **Section/Group: 24AIT_KRG-1A**
Semester: 5
Subject Name: Database ManagementSystem **Subject Code: 24CSH-298**

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than $20,000.
2. **"Average Salary"**: All monthly salaries in the inclusive range [$20,000, $50,000].
3. **"High Salary"**: All monthly salaries strictly greater than $50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

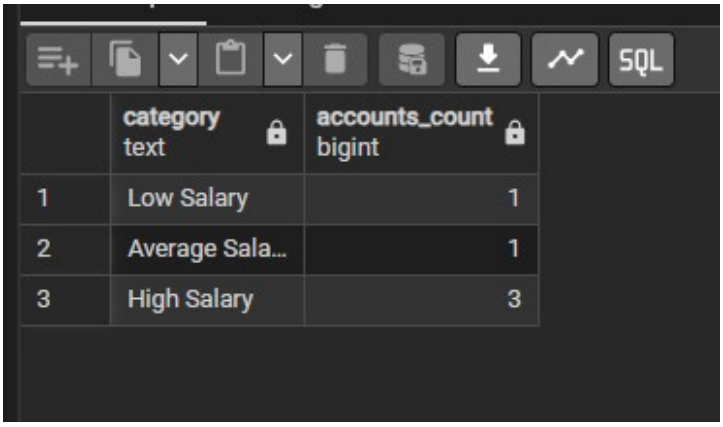
CODE SCREENSHOTS:

```sql
CREATE TABLE Accounts (
    account_id INT PRIMARY KEY,
    income INT
);

INSERT INTO Accounts (account_id, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);
```

EXPERIMENT 1

```sql
INSERT INTO Accounts (account_id, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);



SELECT
    'Low Salary' AS category,
    COUNT(*) AS accounts_count
FROM Accounts
WHERE income < 20000

UNION ALL

SELECT
    'Average Salary' AS category,
    COUNT(*)
FROM Accounts
WHERE income BETWEEN 20000 AND 50000

UNION ALL

SELECT
    'High Salary' AS category,
    COUNT(*)
FROM Accounts
WHERE income > 50000;
```

OUTPUT SCREENSHOTS:



	category text	accounts_count bigint
1	Low Salary	1
2	Average Sala...	1
3	High Salary	3

EXPERIMENT 1

LEARNING OUTCOMES:

1. **Use the CASE statement** to classify data into different categories based on conditions.
2. **Apply aggregate functions** likeCOUNT()tocalculatethenumberof records.
3. **Use GROUP BY** to group records according to salary categories.
4. **Understand conditional grouping** using salary ranges.
5. **Handle edge cases**,suchasdisplayingcategoriesevenwhenthecount is zero.
6. **Write SQL queries** that classify and summarize data efficiently.
7. **Interpret inclusive (>=, <=) and exclusive (<, >) range conditions** correctly.

CONCLUSION:

This SQL problem demonstrates how to classify records into predefined salary categories using the CASE statement and calculate the total number of accounts in each category using the COUNT() aggregate function. It also emphasizes the use of GROUP BY for data summarization and the importance of ensuring that all required categories are displayed, even when a category contains zero records. Overall, the problem strengthens the understanding of conditional logic, aggregation, grouping, and handling edge cases, which are essential skills for writing efficient SQL queries and solving real-world data analysis problems.

EXPERIMENT 1

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1.Create and populate structural tracking tables for Departments, Employees, and Salaries.
2.Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below $30,000.
3.List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

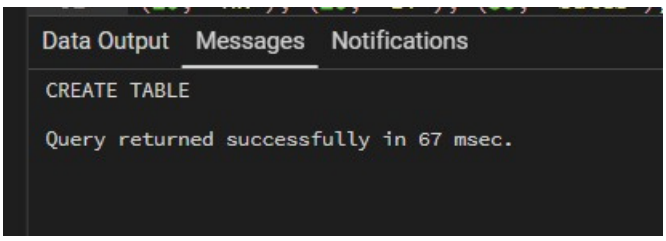
CODE SCREENSHOTS:

```
--(B):
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);

CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
    salary INT NOT NULL
);
```

EXPERIMENT 1

```sql
INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');

INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);

INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);


(i)
SELECT e.name, d.dept_name, s.salary
FROM Employees AS e
JOIN Departments AS d
ON e.dept_id = d.dept_id
JOIN Salaries AS s
ON e.emp_id = s.emp_id;

(ii)
UPDATE Salaries
SET salary = salary * 1.10
WHERE emp_id IN (
    SELECT e.emp_id
    FROM Employees e
    JOIN Departments d
    ON
    e.dept_id = d.dept_id
    WHERE d.dept_name = 'HR'
);

(iii)
SELECT e.name, s.salary
FROM Employees AS e
JOIN Salaries AS s
ON e.emp_id = s.emp_id
WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
```

OUTPUT SCREENSHOTS:



EXPERIMENT 1

Data Output | Messages | Notifications

SQL

	name character varying (50)	salary integer
1	Charlie	80000
2	Alice	55000

LEARNING OUTCOMES:

Design a relational database by creating and populating multiple related tables such as Departments, Employees, and Salaries.

Use SQL DDL commands (CREATE TABLE) to define the database schema.

Insert records into tables using INSERT INTO statements.

Perform multi-table joins (INNER JOIN) to retrieve meaningful information from related tables.

Update records using the UPDATE statement with conditions to modify salary values.

CONCLUSION:

This Employee Management System task provides practical experience in designing and managing a relational database using SQL. It covers the complete database workflow, including table creation, data insertion, multi-table joins, record updates, deletions, and analytical queries using aggregate functions and subqueries. By completing this task, learners gain a strong understanding of fundamental SQL operations and develop the skills required to manage real-world employee and payroll databases efficiently.

EXPERIMENT 1

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.
Every customer must choose **exactly 3 different toppings** to prepare a pizza.
Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.
Requirements
Every pizza must contain **exactly 3 different toppings**.
The same topping **cannot appear more than once** in a pizza.
Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage
Pepperoni, Chicken, Sausage
Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.
Display the total ingredient cost of each pizza.
Sort the output:

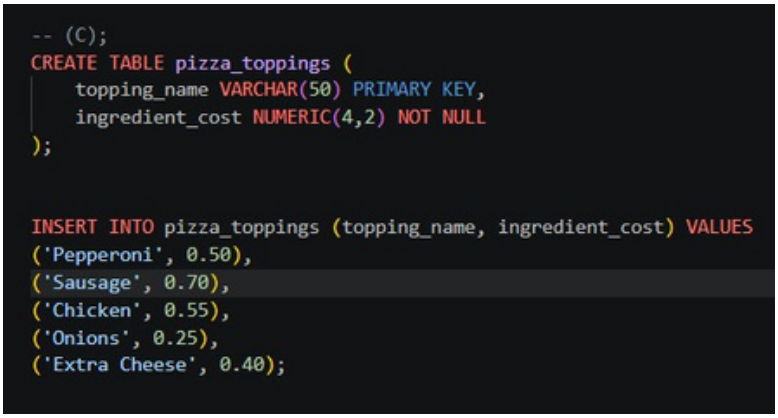
First by **highest total cost**
If two pizzas have the same cost, sort them alphabetically.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

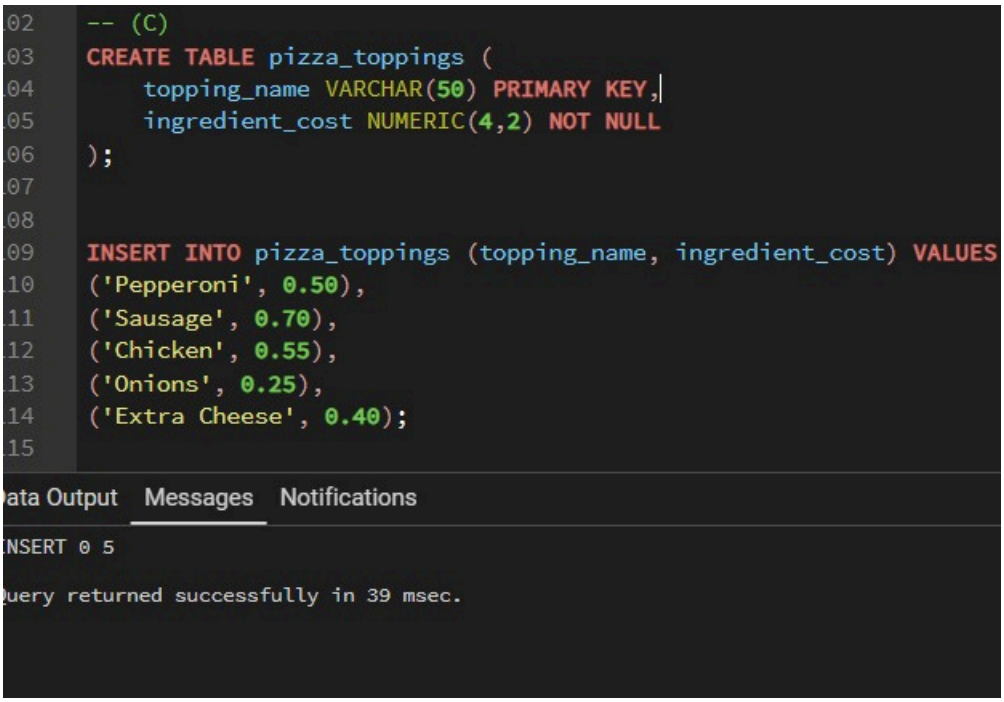
CODE SCREENSHOTS:



```
-- (C);
CREATE TABLE pizza_toppings (
    topping_name VARCHAR(50) PRIMARY KEY,
    ingredient_cost NUMERIC(4,2) NOT NULL
);


INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
('Pepperoni', 0.50),
('Sausage', 0.70),
('Chicken', 0.55),
('Onions', 0.25),
('Extra Cheese', 0.40);
```

EXPERIMENT 1

OUTPUT SCREENSHOTS:



LEARNING OUTCOMES:

□ Generate all possible unique combinations of records using Self Joins.

□ Apply multiple Self Joins to combine rows from the same table while avoiding duplicate combinations.

□ Use comparison operators (<, >) to ensure that the same topping is not selected more than once and that duplicate permutations are eliminated.

□ Calculate aggregate values by summing the costs of multiple toppings to determine the total ingredient cost of each pizza.

□ Concatenate multiple column values into a single comma-separated string for better result presentation.

EXPERIMENT 1

CONCLUSION:

This SQL problem demonstrates how to generate all unique three-topping pizza combinations while preventing duplicate arrangements and repeated toppings. It strengthens the understanding of Self Joins, combination logic, string concatenation, arithmetic operations, and multi-level sorting. By solving this problem, learners gain practical experience in writing advanced SQL queries that generate combinations, compute derived values, and present results in a structured format, preparing them for real-world database querying and interview-level SQL challenges.